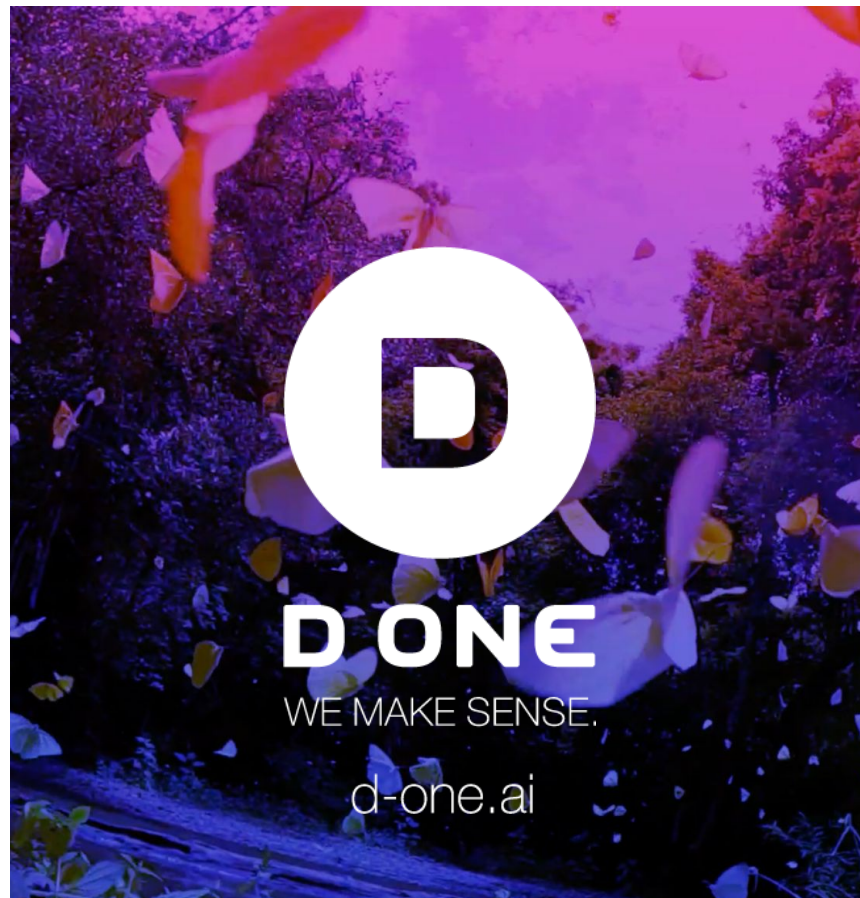


Agentic AI in Practice:

Building and Operating Intelligent
BI Systems for Global Retail

SDS2026 workshop
May 7

Data Driven Value Creation – Never Walk Alone.
To achieve ambitious goals, rely on the best team.



Agenda

First part:

1. Intro (15')
2. Agent Bricks (20')
3. Business use case (10')
4. Hands-on (45')

Second part:

1. LangGraph & DSPy (10')
2. Hands-on (60')
3. Deployment walkthrough (15')
4. Q&A



D ONE

Zurich - Athens - Munich



- Engagements
- Locations

Zurich, since 2005

**Industrializing Data
& AI Value**

**Creative,
entrepreneurial
professionals
with
excellent business
& technical
understanding**

**2021 D ONE in
Athens**

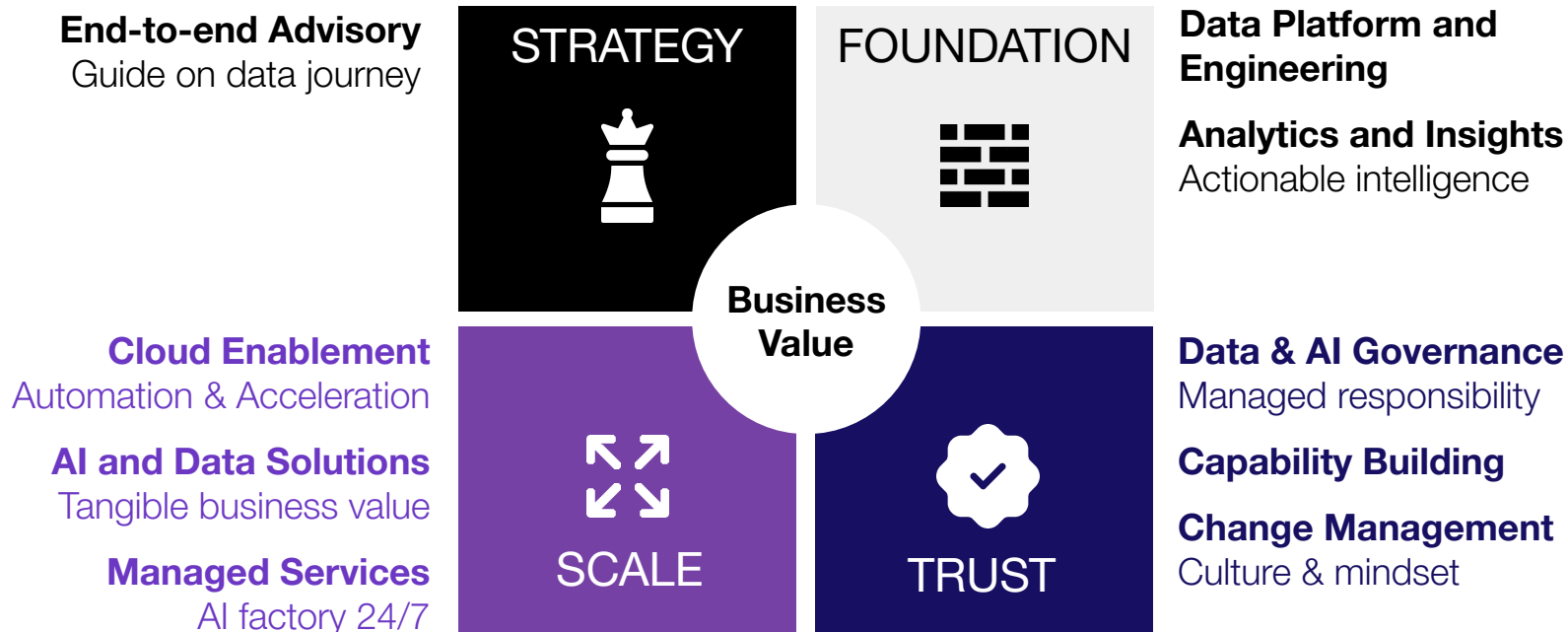
**2025 D ONE in
Munich**

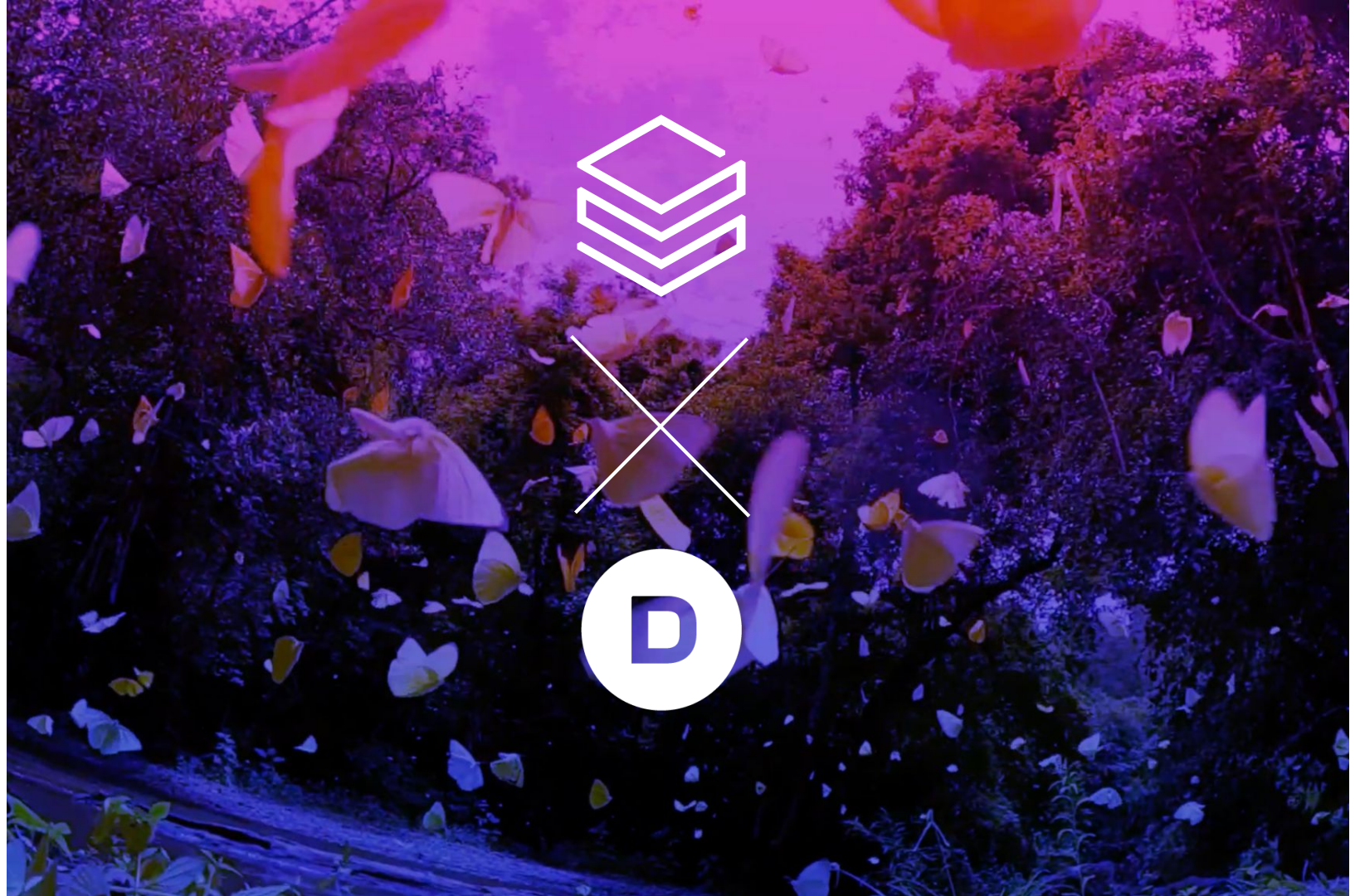
Integrated Delivery
Model

**International ·
Swiss · German ·
Greek clients**

**>1'000 Success
Stories in regulated
industries**

Industrializing Data & AI







Yevgeniy
Ilyin



Andrei
Dmitrenko



Spyros
Cavadias



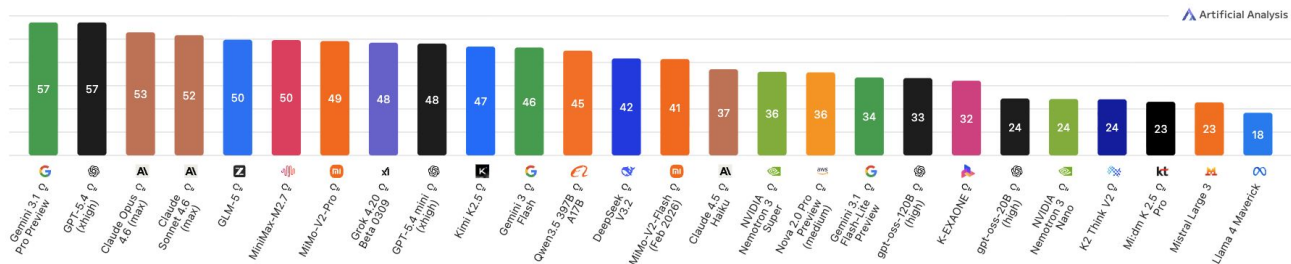
Giorgos
Tzoumanekas

Stone → ... → ChatGPT → Claude Code → OpenClaw

State of Artificial Intelligence

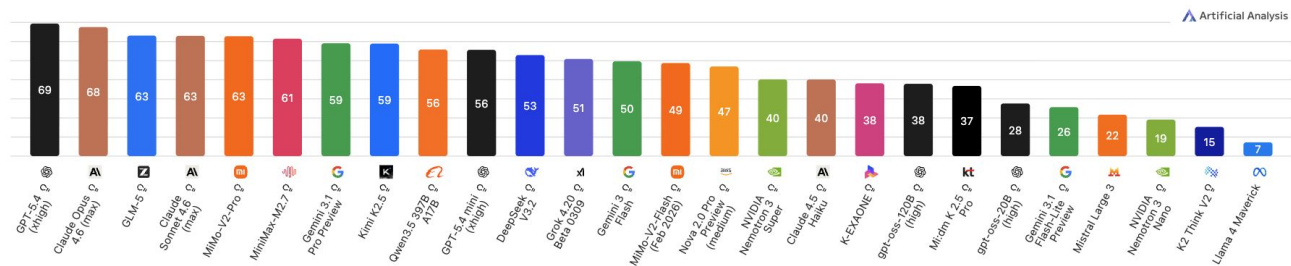
Artificial Analysis Intelligence Index

Artificial Analysis Intelligence Index v4.0 incorporates 10 evaluations: GDPval-AA, τ^2 -Bench Telecom, Terminal-Bench Hard, SciCode, AA-LCR, AA-Omniscience, IFBench, Humanity's Last Exam, GPQA Diamond, CritPt



Artificial Analysis Agentic Index

Represents the average of agentic capabilities benchmarks in the Artificial Analysis Intelligence Index (GDPval-AA, τ^2 -Bench Telecom)



💡 Reasoning models are indicated by a lightbulb icon.

New Opportunities

Multimodal Analysis

Sentiment Analysis

Data Labelling

Chatbots

Information Extraction

Customer Support

AI in ETL

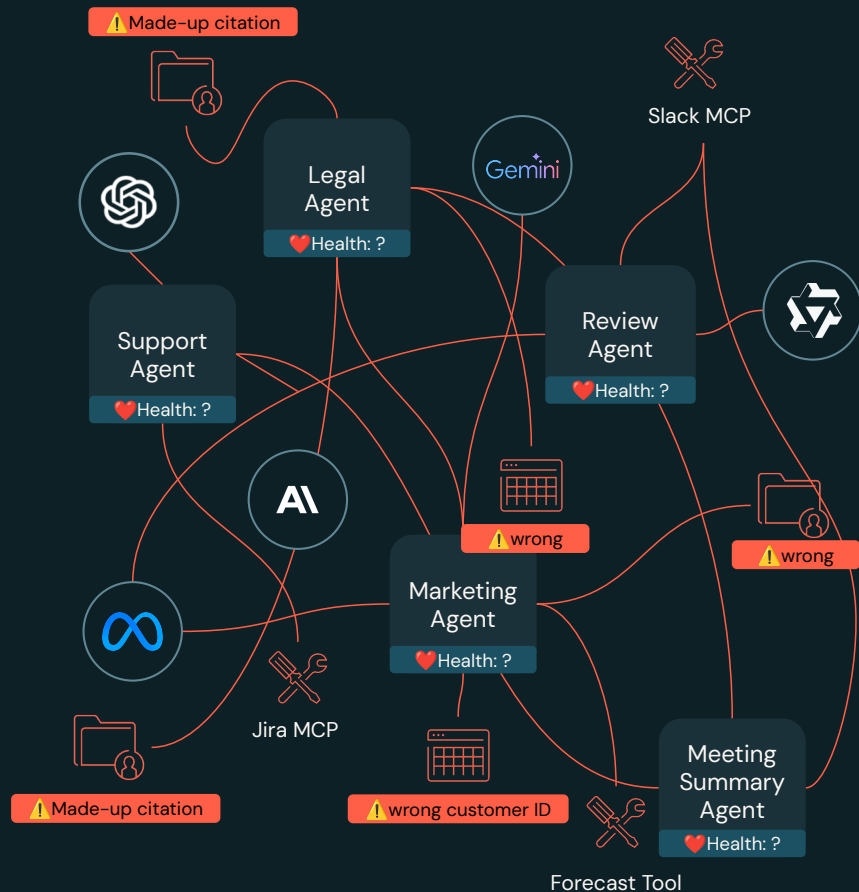
Process Automation

Intent Classification

Phonebots

Agent Bricks





There is **Agent Sprawl**

Low quality reasoning on enterprise data

No visibility or auditability

Too many AI vendors

No way to measure quality



Agent Bricks:

The control plane to
build and manage
enterprise agents



Contextual reasoning



Runs on any model or
framework natively

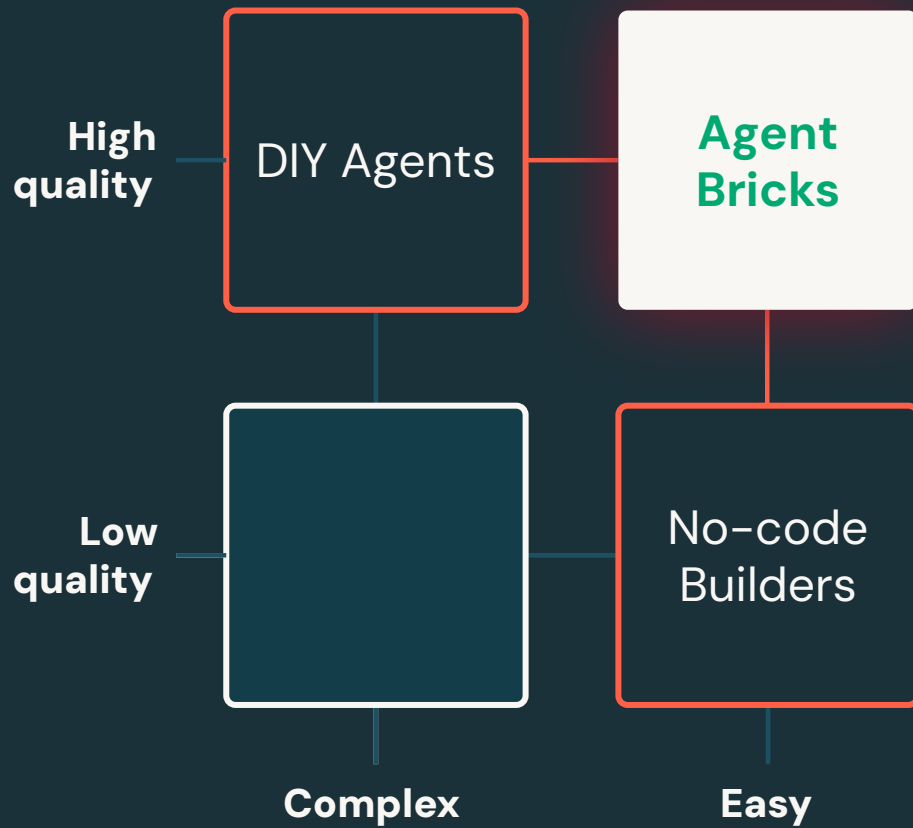


Evaluates & improves quality
over time



Unified catalog
and governance





Learned Semantics

Personalized table interactions

Tool & table popularity

Business Semantics

Metrics View
Agent Metadata

Custom Semantics

Extracted metadata from text
Parsed documents / images



Agent Bricks Contextual Reasoning

Genie: state-of-the-art text2SQL generation

How does the total attributed revenue from AutoML users vary by date?

Analysis (click to view) automl_aggregate

This result is reviewed.

The total attributed revenue from AutoML users varies significantly by date, with some days showing much higher revenue than others. Examples include:

- 2021-02-15: 3.90
- 2021-03-19: 125.08
- 2021-05-28: 335.31
- 2021-06-01: 362.91
- 2021-06-03: 303.28



Contextual reasoning



Runs on any model or framework natively



Evaluates & improves quality over time



Unified catalog and governance



Natively serve all frontier AI models
one contract, any model, no lock-in



**Build custom agents with
any framework**



Across any cloud



Contextual reasoning



Runs on any model or
framework natively



Evaluates & improves
quality over time

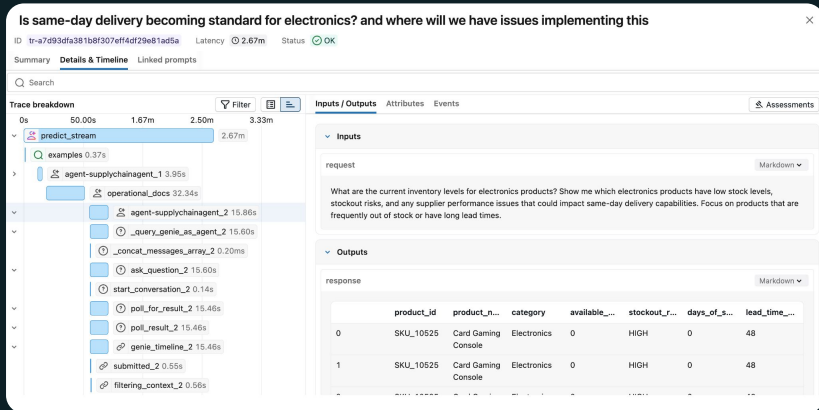


Unified catalog
and governance

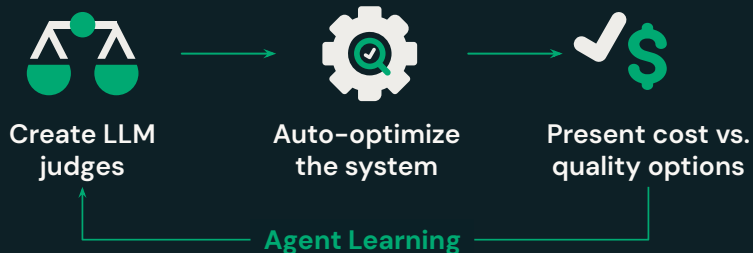


Tracing & Debugging

mlflow



Automatic Improvements



Contextual reasoning

Runs on any model or framework natively

Evaluates & improves quality over time

Unified catalog and governance



Access Control

Discovery

Lineage

Cost Controls

Auditing

Secure Open
Data Sharing

Business
Semantics

Quality
Monitoring



Unity Catalog



Agents

MCP Servers

Models

Skills

Data

External Agents



Contextual reasoning



Runs on any model or
framework natively



Evaluates & improves
quality over time



Unified catalog
and governance





Agent Bricks
Production AI agents



AI/BI
Agentic business
intelligence



Custom Apps
Secure data and AI apps

And more...

Agent Platform

Contextual Reasoning Agents

Knowledge Assistant

Supervisor Agent

Documents Agent

AI Functions

Developer Platform

Agent Orchestration

Runtime  *mlflow*  

Memory

Model
Capacity  Gemini  AI

AI Governance

Agent/Skill/MCP Registry

AI Gateway

Agent Observability

Managed OAuth Apps



Agent Bricks GA

GA



Build Custom Agents

Agent Framework

AI Gateway

Foundation Model APIs

MLflow Evaluation

GA



Build Agents with Enterprise Data

Genie

Knowledge Assistant

Agent Supervisor

GA



Build Document Processing Agents

AI Parse Document

AI Extract

Other AI Functions



Agent Bricks

What's Coming



Memory and
big data
context
management



Unified
catalog for
external
agents, MCP,
& skills



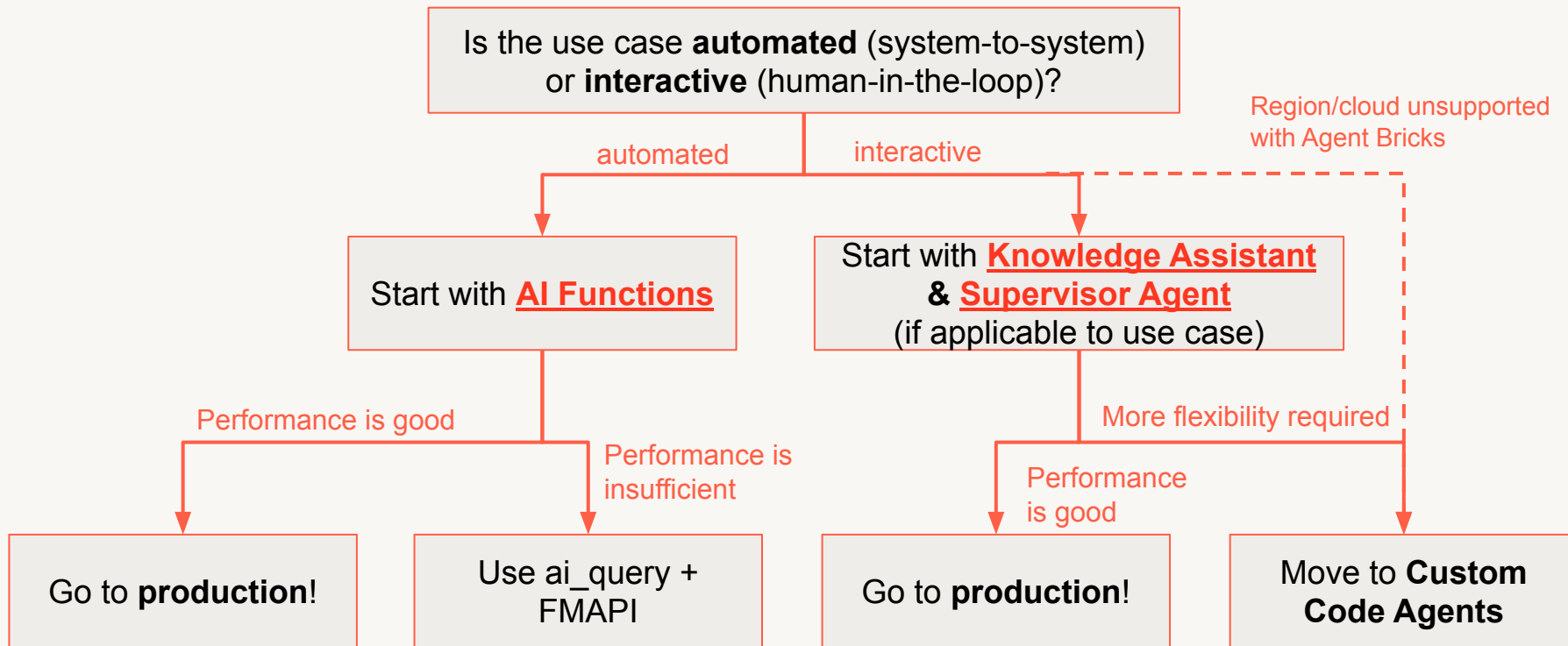
Agents
deployed
as Apps



Claude
Code Skills
for agent
evaluation &
deployment

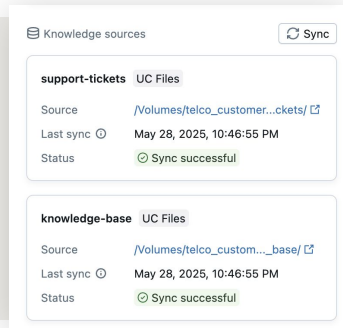


Use Case Guide



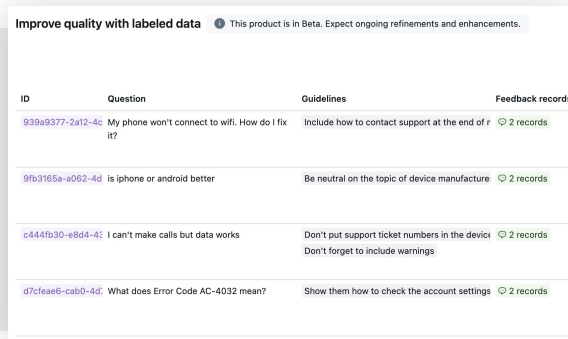
Knowledge Assistant

Use Knowledge Assistant for **the answers you need**



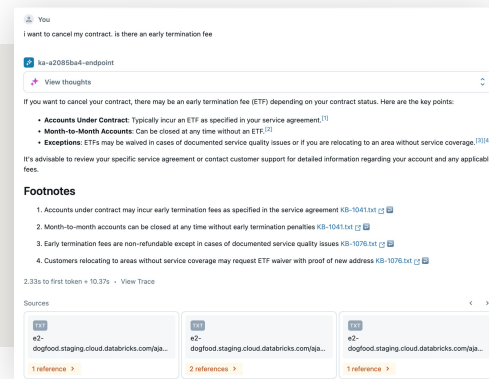
**End-to-end ingestion
handled for you**

File parsing, ingestion,
chunking, and embedding all
handled for you



**Agent learning & evaluation
built-in**

Gather feedback from SMEs &
improve the model over time



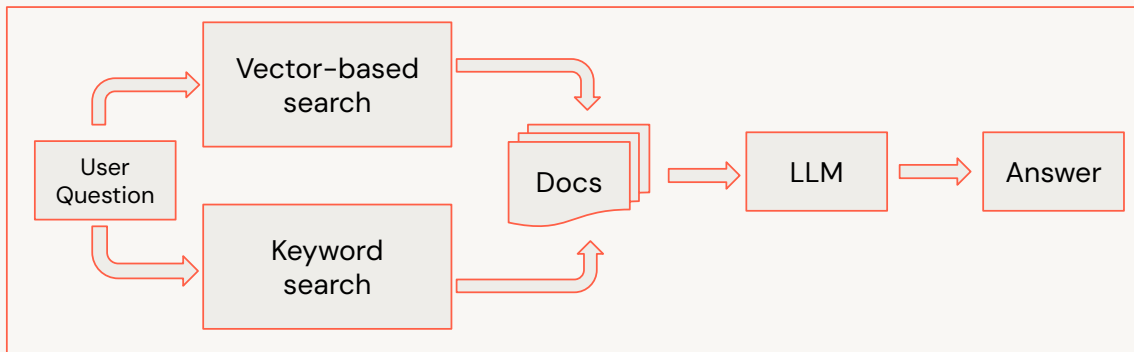
**Ready to scale endpoint
out-of-the-box**

Easily embed your endpoint
wherever you need it – including a
Databricks App



RAG vs Knowledge Assistant: Architecture

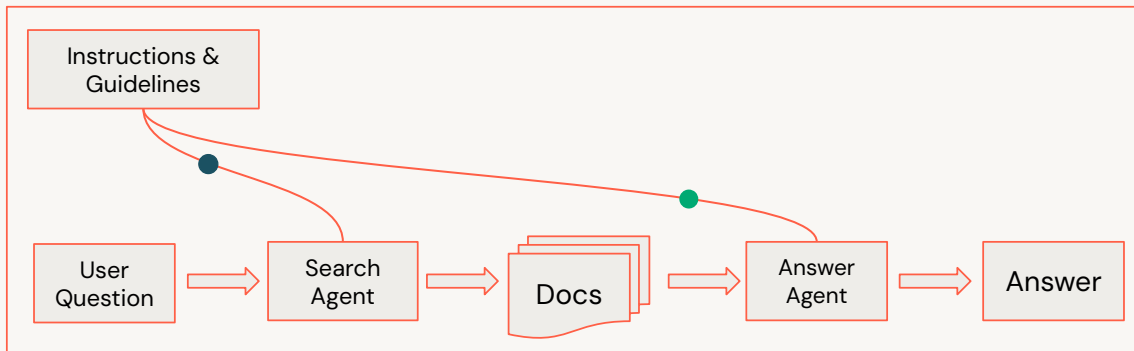
RAG



Limited search quality.

Missing freshness, metadata, etc.

Knowledge Assistant



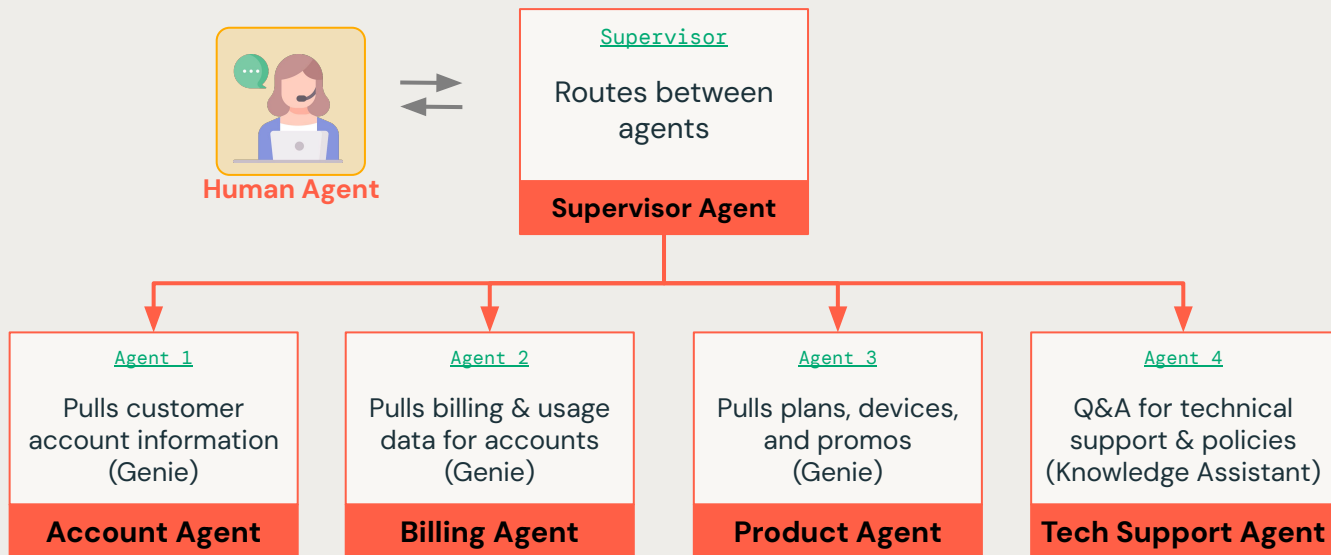
Conversational search with superior quality.

Powered by an Instructed Retriever architecture

Supervisor Agent

Reference Multi-Agent Architecture

A support team copilot leveraging Genie for structured data & Knowledge Assistant for unstructured data



Supervisor Agent

Design an AI system with Genie, agents, and tools

Use endpoints for agents

Endpoints from Agent Bricks (Knowledge Assistant included), or with custom code

Leverage Genie for SQL

Ask questions in natural language and get answers immediately

Include tools to take action

Use MCP servers (Databricks or your own) so your agent can take action

The screenshot displays the configuration interface for a 'telco-multi-agent-0607' agent. The interface is divided into several sections:

- Basic Info:** Shows the agent's name 'telco-multi-agent-0607', its endpoint 'mas-728a08bc-endpoint', and its MLflow experiment 'mas-728a08bc-dev-experiment'.
- Description:** Provides a detailed description of the agent's capabilities, including routing questions about billing, technical support, and customer accounts.
- Configure Agents:** Lists other agents available for configuration, such as 'telco-tech-support-0603' and 'agent-billing'.
- Test your Agent:** A section for testing the agent's performance, including a 'Test your Agent' button and a 'Provide feedback' link.
- Output:** Displays the output of a test run, showing a JSON response for a request about a device not turning on.
- Footnotes:** A section for additional information or references.

The interface also includes a 'Cancel' button and an 'Update Agent' button at the bottom right.



Supervisor agent

Orchestrates multiple tools or agents to work optimally together

Example use cases:

- Generate insights across both unstructured and structured data spanning across different departments
 - Integrates domain-specific agents specialized for unstructured and structured data together
- A healthcare platform that follows up with action steps and resources
 - Orchestrates role-specific agents for both the healthcare providers and patients



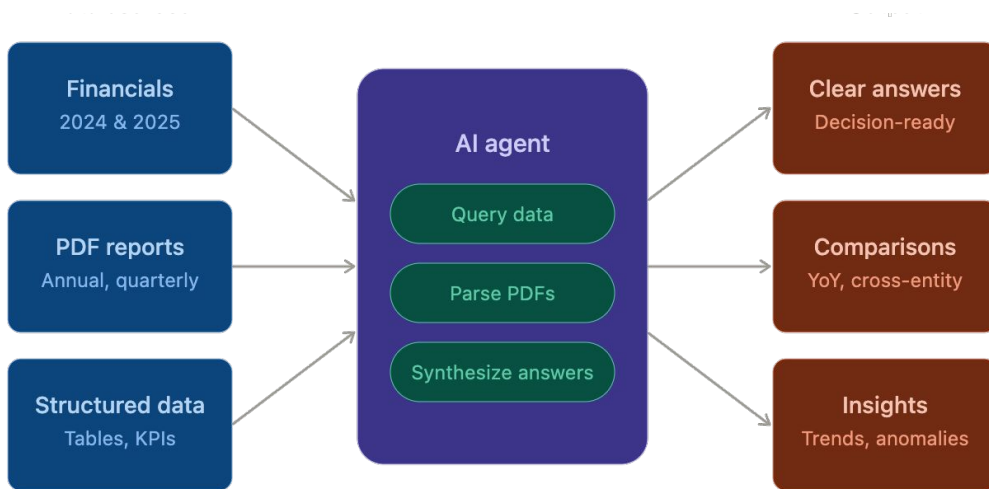
Let's introduce our use case

Use case

You are a financial analyst working for the reporting function, reporting to the Group CFO. You need to quickly answer business questions using multiple data sources. Your available data sources are Financial Statements and Reports from 2024 & 2025. All data used in this use case (and more) are available freely [here](#).

Your goal is to build an AI agent that can:

- Query structured financial data
- Understand financial reports (PDFs)
- Deliver clear, decision-ready answers



Hands-on

Hands-on: Setup

Access Databricks Workspace

01

Open link and connect with credentials: <https://www.tinyurl.com/0590c42f>

Username: agentbricks-X@ms.d-one.ai (X = user number provided to you)

Password: Bricks26!

Requires a 2FA Microsoft Authentication app. We recommend using Google Chrome web browser.

02

Initialize Git Workspace

In the sidebar navigate to **Workspace** > **Create** > **Git Folder**

03

Clone Workshop Repository

Paste the link and click **Create Git folder**:

`https://github.com/d-one/sds26\_agentbricks\_workshop.git`



Hands-on: First part (I)



Navigate to **1.1 Building a Financial Intelligence Agent** and follow step by step.

Checkpoints



You have created **Knowledge Assistant** and tested it out



You have created **Genie** and tested it out



Connected them with the **Supervisor** and tried out the notebook queries



Hands-on: First part (II)



Navigate to **1.2 Evaluation of Financial Intelligence Agent** and follow step by step.

Checkpoints



You have found your **supervisor endpoint**

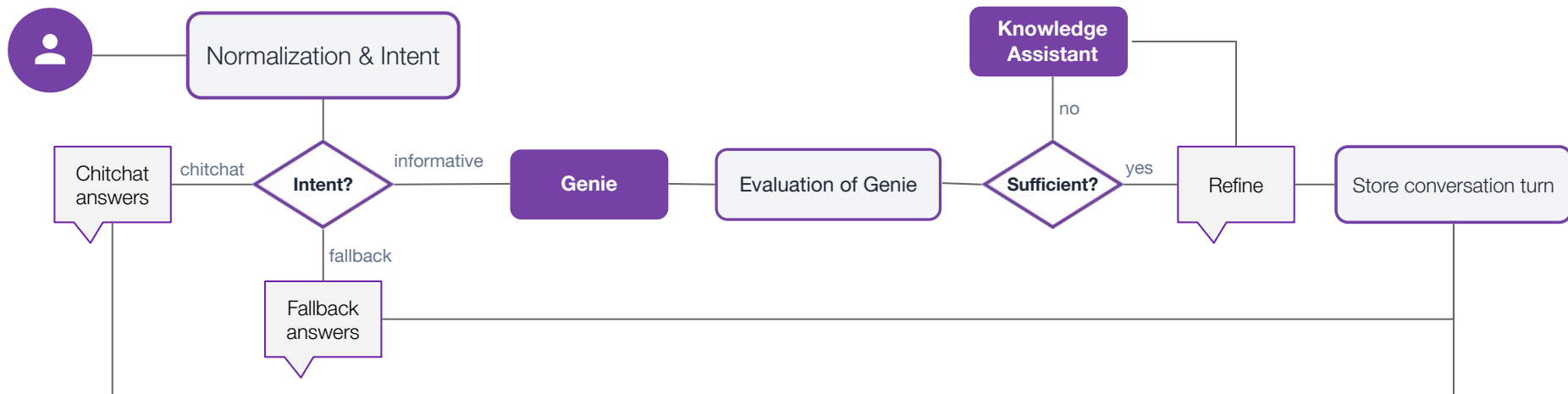


You have run the **evaluation** and can see the results in **Experiments**



Agent architecture

Agent Architecture Overview



1. Normalize & Route

The Normalizer after normalizing the query will have to route it based on intent.

2. Genie

If the normalizer decides the question is informative, the Genie tool is used

3. Supervisor

The supervisor decides if Genie's response is enough. If not it calls the Knowledge Assistant

4. Refine

Refiner polishes the final answer that comes from Genie only or Genie and KA.

5. Update history

The history is updated with the conversation turn no matter which tool was called.



LangGraph, Pydantic & DSPy



Problem Framing

Challenges in Agentic Workflows:

- **Multiple tools:** Integrating Genie, RAG, and various LLMs seamlessly.
- **Multiple steps:** Managing routing, complex decisions, and iterative refinement.
- **Non-linear logic:** Moving beyond simple linear pipelines to cyclic graphs.

👉 **We need a way to orchestrate reasoning**

LangGraph Solution

LangGraph, created by LangChain, is an open-source framework for **stateful, multi-step workflows**.

Key Capabilities:

- Connect multiple steps (nodes)
- Add decision points (conditional routing)
- Maintain memory (state)
- Build complex nonlinear flows



Unpredictable LLMs

Without structure, outputs vary:

- Conversational filler (“I think...”)
- Missing required data fields
- Inconsistent or wrong formats
- Hard-to-parse responses

👉 Fragile and hard to automate

Pydantic Solution

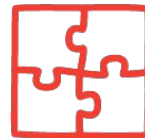
The industry-standard data validation library for Python.

Key Capabilities:

- **Fixed Response Format:** Enforces schema consistency.
- **Data Validation:** Ensures all required fields are present.
- **Machine-Readable:** Seamlessly converts LLM output to objects.

DSPy

A framework to **structure and optimize prompts programmatically**



Key Capabilities

- Define prompts as components (**Signatures**)
- Enforce structured outputs (with Pydantic)
- Reusable and testable prompting
- Better control than raw prompting

More Capabilities

- Agent evaluation framework
- Automated prompt optimization
- Systematic approach to agent improvement

But these are outside of our scope today.

Hands-on

Hands-on: Second part (I)



Navigate to **2.1 LangGraph Agent** and follow step by step.

Checkpoints



You have created **each Signature** and tested each node



You have created the **workflow** without errors



You have tried out the queries with the agent **inside the notebook**



From Notebook to Endpoint

The ResponsesAgent Subclass

Key benefits:

- Support for **multiple output messages**, including intermediate **tool outputs**
- Built for **multi-agent scenarios** and complex orchestration
- Native **MLflow compatibility** for logging, tracing, and serving
- Seamless integration with OpenAI's responses client and downstream UIs



Wrapping the Agent

The wrapper handles how users interact with the agent.

`load_context()`

- Runs once when the endpoint starts
- Loads the LLM
- Compiles the LangGraph

`predict_stream()`

- Extracts user query + conversation history
- Invokes the LangGraph agent
- Streams the response back

`predict()`

- Wraps streaming output into single response
- Used when streaming is not supported

```
def load_context(self, context):  
    self.agent = get_compiled_graph()
```

```
def predict_stream(self, request):  
    final_state = self.agent.invoke(initial_state)  
    yield self.create_text_delta(...)
```

```
def predict(self, request):  
    return ResponsesAgentResponse(output=[...])
```



Logging the Agent

We log the agent using [MLflow](#), which ensures everything is reproducible.

What should we log?

Our Agent Class: `AvoltaAgent`

Our resources: Passthrough for Genie, models, tables, etc.

All dependencies: Pinned with their versions

An input example: For testing and schema purposes

```
mlflow.pyfunc.log_model(  
    python_model=AvoltaAgent(),  
    resources=resources,  
    artifact_path="agent"  
    pip_requirements=[...]  
    input_example={"input":  
        [{"role": "user", "content": "What is the revenue?"}]  
    }  
)
```

Registering the Agent

Once logged, we register the model in **Unity Catalog** for centralized management.

Why register it?

- **Versioning**: Track iterations (v1, v2, v3) seamlessly.
- **Reliability**: Safe updates without breaking production.
- **Governance**: Centralized access control and auditing.
- **Lifecycle**: Enables easy promotion and rollback.

Code Example:

```
model_version = mlflow.register_model(  
    model_uri=f"runs:{run_id}/agent",  
    name=UC_MODEL_NAME  
)
```



Deploying the Agent

We deploy the agent as a **real-time serving endpoint**.

Accessibility & Scalability

The agent is now live and accessible through:

- AI Playground
- Review App (shareable UI for feedback)
- External applications

Scale to Zero (!)

- Saves cost when idle
- Automatically restarts on request

```
agents.deploy(  
    model_name=MODEL_NAME,  
    model_version=model_version.version,  
    endpoint_name="avolta_agent",  
    scale_to_zero=True  
)
```

Hands-on

Hands-on: Second part (II)



Continue with **2.1 LangGraph Agent** and follow the deployment step by step.

Checkpoints



You have **deployed** the agent without errors



You have **tested out** your agent endpoint



Hands-on: Second part (III)



Navigate to **2.2 Evaluation of LangGraph Agent** and follow step by step.

Checkpoints



You have found your **agent endpoint**



You have ran the **evaluation** and can see the results in **Experiments**



Conclusion

Conclusion

First, you have built a **multimodal agentic RAG** with Agent Bricks.

Then, you have employed an **evaluation framework with MLflow**.

Later, you have replicated the agentic RAG architecture with **LangGraph**.

Finally, you have successfully **deployed and evaluated** it.

Now you know how to build **production-ready agentic solutions** on Databricks! 🎉

Thank you for attending!